ASYNC-001: FastAPI BackgroundTasks Behavior Review

Status: Waiting for Review

Parent Ticket: HTTP-001

Date: 2026-08-12

---

1. Overview

This document reviews the behaviour of background tasks in the current FastAPI codebase.

Current Implementation Strategy:

Fire-and-forget pattern utilising FastAPI's native `BackgroundTasks` class. All background functions are standard synchronous `def` functions. FastAPI automatically executes these functions in a separate thread pool (`ThreadPoolExecutor`) managed by Starlette. There is no external queue (e.g., Celery, Redis), no persistent job storage, and no supervision tree. Tasks are referenced in-memory and execute within the application process.

Lifecycle rule: A task persists until:

- It completes successfully.
- It raises an unhandled exception (dies silently).
- The Uvicorn process is terminated, the server crashes, or the host machine shuts down.

---

2. Detailed Findings

2.1. Cancellation

Aspect	Detail
Current Behaviour	No explicit cancellation logic is implemented.

Aspect	Detail
On Client Disconnect	If the HTTP client disconnects, the background task continues running to completion. FastAPI `BackgroundTasks` does not propagate cancellation signals from the request lifecycle.
Manual Cancellation	Not supported. There is no mechanism to cancel a specific running background task via an API or administrative interface.

Risk:

Orphaned tasks can consume CPU, memory, and database connections long after their results are no longer required, potentially leading to resource exhaustion under high load.

2.2. Lifecycle

Aspect	Detail
Start	Tasks are added to the background task queue and executed after the HTTP response is sent to the client.
Running State	They run as standard threads within the application's default thread pool.
Completion	Normal completion: the thread terminates and resources are released.
Shutdown	On application shutdown (`SIGTERM` , `SIGINT` , or process kill), all running background threads are abruptly terminated. No graceful shutdown timeout or `atexit` handler is configured.
Machine Restart	All in-memory tasks are destroyed. There is no persistence layer to recover failed or in-progress jobs.

Risk:

Critical operations (e.g., database migrations, invoice generation, email dispatching) can be interrupted during deployments or server restarts, leading to data inconsistency or missed operations.

2.3. Exceptions

Aspect	Detail
Current Behaviour	No `try/except` block wraps the background task's core logic.
Failure Mode	If an unhandled exception occurs inside the background function, the thread terminates silently. The exception is logged to the console via

Aspect	Detail
	the Uvicorn/FastAPI default logger, but the original HTTP caller never receives any feedback.
Retry Logic	None implemented. If the task fails, it is not retried.
Monitoring	No custom alerts or metrics are emitted when a background task fails.

Risk:

Silent failures make debugging extremely difficult. If this task handles side effects (e.g., updating a status column in a database, calling a third-party API), the system will be left in an inconsistent state without the operator's knowledge.

2.4. Blocking Operations and Threading Considerations

Aspect	Detail
Event Loop Impact	As `BackgroundTasks` executes synchronous `def` functions via a thread pool, the main asyncio event loop remains entirely unblocked. The web server can continue serving incoming requests without interruption.
Thread Pool Saturation	The default thread pool size is typically limited to 40 threads (configurable via `threads_per_worker`). Under heavy concurrent background task loads, the pool can become exhausted, causing subsequent background tasks to queue and introducing latency.
CPU-Bound Operations	Standard Python threads are bound by the Global Interpreter Lock (GIL). Heavy CPU-bound operations executed within a background task will not release the GIL and may starve other threads in the pool. Such operations are better suited for multiprocessing.
Thread Safety	Background tasks execute in separate threads. Shared resources (e.g., database session objects, application caches, global variables) must be handled with thread-safe practices. Using ORM session objects across threads without proper isolation can lead to race conditions and connection corruption.

Recommendation:

- Avoid CPU-intensive computations inside background tasks. Defer them to external dedicated services or multiprocessing workers.
- Ensure each background task uses isolated database sessions or connection objects to prevent cross-thread interference.

- Monitor thread pool utilisation and adjust `threads_per_worker` accordingly if background tasks become frequent.

---

3. Risk Summary

Risk Area	Severity	Notes
Task loss on deployment or restart	High	No graceful shutdown or persistence means in-progress jobs are lost during code updates.
Silent exception failures	Medium	Logs exist, but no retries or alerts; requires manual log monitoring.
Thread pool exhaustion	Medium	Under sustained high load, the default thread pool may become saturated, delaying background task execution.
Thread safety violations	Medium	Shared database sessions or mutable global state accessed across threads can cause unpredictable failures.
CPU-bound operations blocking thread pool	Medium	Heavy computations within threads can degrade overall thread pool performance.

---

4. Conclusion

The current implementation leverages FastAPI's `BackgroundTasks` with synchronous functions executed in a thread pool. This approach successfully avoids blocking the asyncio event loop. However, it remains a basic fire-and-forget pattern that lacks cancellation, graceful shutdown, retry logic, and persistence. It is suitable only for short-lived, idempotent, non-critical I/O-bound operations (e.g., sending a confirmation email, clearing a cache). It is not suitable for business-critical workflows that require durability, monitoring, or exactly-once semantics. Additionally, thread safety must be explicitly managed when accessing shared resources.

---

5. Recommendations

5.1. Short-Term Mitigations

- Wrap all background task logic in a `try/except` block with structured logging and a dead-letter fallback to record failures.

- Implement graceful shutdown handlers that wait for remaining background tasks to complete (or timeout) during `SIGTERM` before worker termination. This can be achieved using `BackgroundTasks` in conjunction with `asyncio` shutdown hooks or by maintaining a thread-safe task registry with completion tracking.
- Audit all background tasks to ensure database sessions and application resources are thread-isolated and not shared across concurrent threads.
- Profile the thread pool size (`threads_per_worker` in Uvicorn) against expected concurrent task volumes to prevent queue backlogs.

5.2. Future Improvement: Celery Integration

For long-term scalability and reliability, migrating critical background jobs to a dedicated distributed task queue such as Celery is strongly recommended.

Key benefits of adopting Celery:

- **Persistence and Durability:** Tasks are stored in a broker (e.g., Redis, RabbitMQ) and survive application restarts. In-progress tasks can be re-queued or recovered after worker failures.
- **Retry Logic:** Native support for exponential backoff retries with configurable max attempts, reducing transient failure rates (e.g., third-party API rate limits, network timeouts).
- **Observability:** Integration with monitoring tools (Flower, Prometheus, Datadog) provides real-time visibility into queue depth, task success rates, and execution latency.
- **Resource Isolation:** Background work runs in separate worker processes, completely decoupled from the web server. This allows horizontal scaling of workers independently of API replicas and isolates CPU-bound workloads using separate process pools.
- **Scheduled Tasks:** Celery Beat enables native support for cron-like scheduled jobs without requiring external schedulers.
- **Result Backend:** Provides optional storage of task results (e.g., in Redis or a database) for later retrieval by the client via a polling mechanism.

Planned migration approach:

1. Introduce Celery with Redis as the broker in a development environment.

2. Refactor existing fire-and-forget endpoints to enqueue tasks via `task.delay()` or `task.apply_async()`.
3. Implement a status endpoint (`/task/{task_id}`) for clients to poll for completion, replacing the current fire-and-forget model where appropriate.
4. Deploy Celery workers as a separate service alongside the FastAPI application, ensuring zero downtime during deployments via worker rolling restarts.